

Dự đoán mức lương kỳ vọng dựa trên bản mô tả công việc

*Tiểu luận cuối kỳ — Khoa học Dữ liệu —
Năm học 2025-2026*

Nhóm 09

Table of Contents

**TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN**

**TIỂU LUẬN CUỐI KỲ
HỌC PHẦN: KHOA HỌC DỮ LIỆU**

**DỰ ĐOÁN MỨC LƯƠNG KỲ VỌNG
DỰA TRÊN BẢN MÔ TẢ CÔNG VIỆC
(JOB DESCRIPTION)**

Nhóm: 09

Họ và tên sinh viên

MSSV

Lớp học phần

Võ Trần Anh Quân (nhóm trưởng)

102230131

...

Phạm Hữu Tuân

102230139

...

Trần Việt Toàn

102230137

...

ĐÀ NẴNG, 05/2026

TÓM TẮT

Báo cáo trình bày quá trình xây dựng một mô hình học máy dự đoán mức lương kỳ vọng (triệu VNĐ) cho bản tin tuyển dụng tiếng Việt, dựa trên 606,878 bài đăng công khai giai đoạn 2022-2026. Thách thức chính nằm ở việc trường lương trong dữ liệu gốc là văn bản tự do với nhiều biểu diễn không nhất quán (khoảng VNĐ, USD, đơn vị tháng, giá trị đơn lẻ, sentinel cực lớn). Nhóm thiết kế pipeline 4 giai đoạn gồm EDA, làm sạch quy tắc, trích xuất 32,107 đặc trưng (numeric + categorical + multi-label + TF-IDF text), và huấn luyện 5 nhóm mô hình (Baseline, Ridge, Lasso-SGD, LightGBM tinh chỉnh Optuna, Ensemble). Mô hình tốt nhất — LightGBM — đạt **MAE = 2.20 triệu, RMSE = 4.10 triệu, $R^2 = 0.63$** trên tập kiểm thử độc lập 114,773 bản tin (cải thiện 44% so với baseline). Hai phát hiện đáng chú ý: (1) năm đăng gần như không đóng góp dự đoán (Δ MAE chỉ +0.005 khi loại bỏ), (2) ensemble không cải thiện so với LightGBM đơn lẻ do lỗi hai mô hình tuyến tính tương quan rất cao ($r = 0.985$), dẫn đến quyết định cuối cùng dùng LightGBM làm mô hình triển khai.

BẢNG PHÂN CÔNG NHIỆM VỤ

Sinh viên thực hiện

Các nhiệm vụ

Tự đánh giá

Võ Trần Anh Quân 102230131 (nhóm trưởng)

– Thiết kế kiến trúc pipeline tổng thể, quản lý mã nguồn và môi trường (uv, pyproject)
– Notebook 03_Feature_Engineering.ipynb: pipeline 32k đặc trưng (TF-IDF 4 cột + OHE + multi-label + numeric) – Notebook 04_Modeling.ipynb: Baseline, Ridge, Lasso-SGD, LightGBM với Optuna 15-trial Bayesian, year ablation, ensemble – Soạn báo cáo PDF & slide cho mảng Feature Engineering và Modeling – Tổng hợp và biên tập báo cáo + slide cuối cùng

Đã hoàn thành

Phạm Hữu Tuấn 102230139

– Notebook 01_EDA.ipynb: khảo sát thăm dò, trực quan hoá phân phối lương, năm kinh nghiệm, tỉnh thành, ngành nghề – Phát triển 3 hàm parse (lương, năm kinh nghiệm, tỉnh thành) làm cơ sở cho stage 2 – Soạn báo cáo PDF & slide cho mảng Mô tả & Trực quan hoá dữ liệu

Đã hoàn thành

Trần Việt Toàn 102230137

– Notebook 02_Cleaning.ipynb: parse expected_salary qua 4 regex pattern, parse years_exp với rule junk detection, parse province regex 2 lớp, chuẩn hoá industries_list multi-label – Soạn báo cáo PDF & slide cho mảng Tiền xử lý & Làm sạch dữ liệu

Đã hoàn thành

1. Giới thiệu

1.1. Bối cảnh và bài toán

Thị trường tuyển dụng trực tuyến Việt Nam tăng trưởng nhanh trong 5 năm gần đây, nhưng **trường lương trong bản tin tuyển dụng rất không nhất quán**: cùng nội dung

có thể được ghi là “15-20 triệu”, “Từ 15.000.000 VND”, “Theo thỏa thuận”, “Up to 1500 USD”, hoặc “Chưa cập nhật”. Sự không nhất quán này gây khó khăn cho cả ứng viên (khó so sánh giữa các tin) lẫn nhà phân tích thị trường lao động.

Bài toán đặt ra: **dự đoán mức lương kỳ vọng của bản tin tuyển dụng dựa trên toàn bộ thông tin còn lại trong tin đó**. Cụ thể:

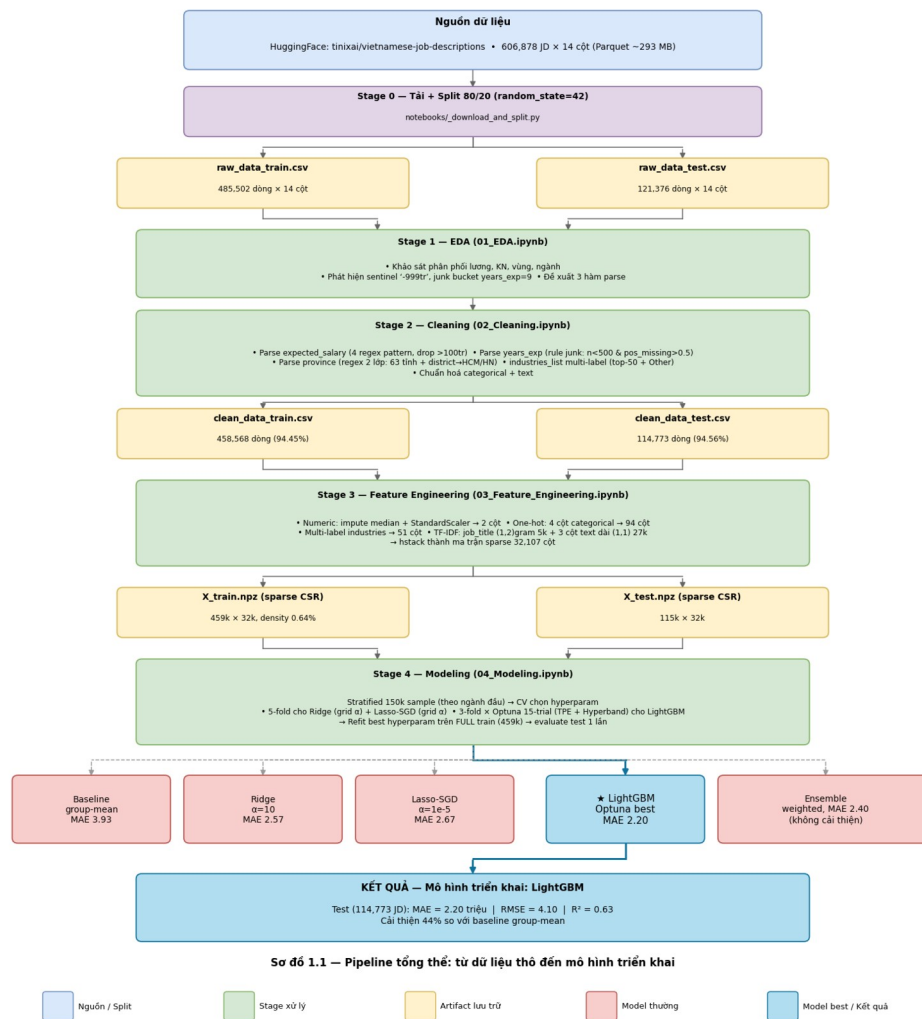
Đầu ra (target): biến liên tục $\text{expected_salary} = (\text{lương_min} + \text{lương_max}) / 2$, đơn vị triệu VNĐ. Ví dụ “23-28 triệu” $\rightarrow 25.5$. Đây là bài toán hồi quy.

Đầu vào: 13 cột còn lại (chức danh, công ty, địa điểm, ngành nghề, kinh nghiệm, học vấn, cấp bậc, mô tả công việc, quyền lợi, yêu cầu chi tiết, năm đăng, ID). Cột salary được dùng để tạo nhãn, không làm feature (tránh leak).

Độ đo: ưu tiên **MAE** ở thang triệu VNĐ (dễ giải nghĩa), kèm RMSE và R^2 để bổ sung góc nhìn.

1.2. Phương pháp và cấu trúc báo cáo

Pipeline gồm 4 giai đoạn — **EDA** $\rightarrow$ **Cleaning** $\rightarrow$ **Feature Engineering** $\rightarrow$ **Modeling** — được hiện thực trong 4 Jupyter notebook và đảm bảo tái lập (mọi random dùng `random_state=42`, tham số fit trên train rồi áp dụng nguyên cho test). Sơ đồ 1.1 mô tả toàn bộ luồng xử lý từ dữ liệu thô đến mô hình triển khai cuối cùng.



Sơ đồ pipeline tổng thể: dữ liệu thô từ HuggingFace → split 80/20 → 4 stage xử lý (EDA, Cleaning, Feature Engineering, Modeling) → 5 mô hình thử nghiệm → mô hình triển khai cuối cùng LightGBM (MAE 2.20 triệu trên test).

Báo cáo tổ chức theo template 6 phần: Phần 2 mô tả dữ liệu và kết quả EDA; Phần 3 trình bày trích xuất đặc trưng; Phần 4 đi sâu vào năm mô hình đã thử và phân tích lỗi; Phần 5 tổng kết và đề xuất hướng phát triển; Phần 6 liệt kê tài liệu tham khảo.

2. Thu thập và mô tả dữ liệu

2.1. Thu thập dữ liệu

Bộ dữ liệu **tinixai/vietnamese-job-descriptions** [4] công khai trên Hugging Face Hub, gồm **606,878 bản tin tuyển dụng tiếng Việt** thu thập từ nhiều công việc làm Việt Nam giai đoạn 2022-2026, định dạng Parquet ~293 MB. Nhóm dùng script `notebooks/_download_and_split.py` để tải qua URL công khai, đọc bằng `pyarrow`, và chia ngẫu nhiên thành tập huấn luyện (80%, 485,502 dòng) và kiểm thử (20%, 121,376

dòng) với random_state=42. Tập train vượt xa yêu cầu tối thiểu của đề bài (>1000 mẫu).

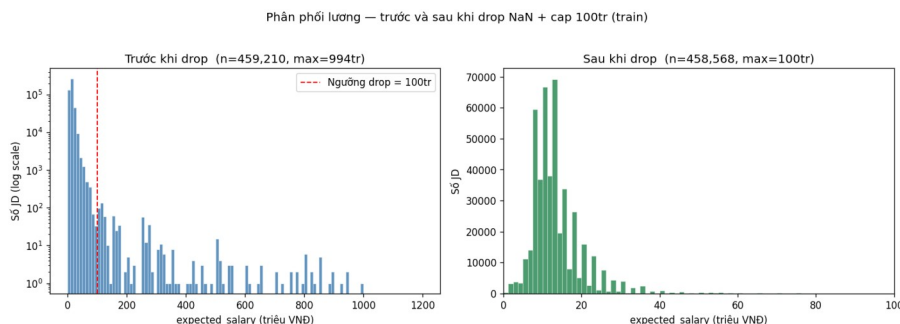
Schema 14 cột: gồm 1 cột mục tiêu (salary — văn bản tự do dùng để parse nhãn), 2 cột numeric (year, ngày hiệu id), 3 cột categorical ngắn (education_level, job_type, job_position), 1 cột multi-label (job_industry), 1 cột vùng (location), 1 cột kinh nghiệm (experience_level), và 5 cột text dài (job_title, company_name, job_description, requirements, benefits).

2.2. Mô tả và trực quan hoá dữ liệu

2.2.1. Biến mục tiêu

Trường salary là văn bản tự do, được parse bằng **4 regex pattern**: vnd_range_dotted (“15.000.000 - 20.000.000 VND”), vnd_range_month (đơn vị MONTH), vnd_single_dotted (giá trị đơn lẻ), và usd_range (quy đổi tỉ giá 25,000 VND/USD). Sau parse, **94.58%** mẫu trên train cho ra expected_salary hợp lệ; 5.42% còn lại là “*Theo thoả thuận*”, “*Đang cập nhật*” — bị loại.

Một phát hiện quan trọng: **642 dòng** (0.13% train) có lương từ vài trăm triệu đến vài trăm tỉ, hầu hết là sentinel 0 - 999.000.000 VND mà publisher dùng để đánh dấu “lương không giới hạn”. Nhóm chọn **loại bỏ hoàn toàn các dòng > 100 triệu** thay vì cap, vì cap sẽ tạo bias rằng “tin cao cấp đều ở mức 100 triệu” — một dạng leak thông tin xử lý.



Phân phối expected_salary trên train trước và sau loại bỏ các giá trị > 100 triệu. Đuôi trái chứa sentinel publisher dùng đánh dấu “lương không giới hạn”.

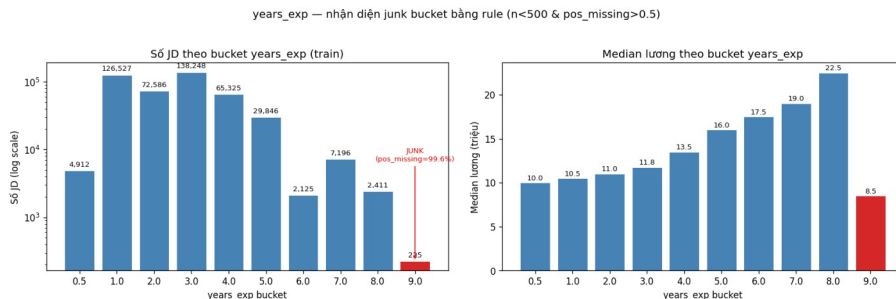
Sau parse và loại junk: tập train còn **458,568 mẫu** (giữ 94.45%), test còn **114,773 mẫu** (94.56%). Thống kê (triệu VNĐ): mean = 13.32, median = 12.00, std = 6.80, min = 0.15, max = 100. Phân phối lệch phải rõ (skew $\approx +2.5$), phần lớn 8-20 triệu, đuôi kéo lên 50-100 triệu cho vị trí cấp cao.

2.2.2. Năm kinh nghiệm và phát hiện junk bucket

Cột experience_level được parse bằng regex `(\d+)\s*năm`; “*Dưới 1 năm*” $\rightarrow$ 0.5; “*Không*”/“*Chưa cập nhật*” $\rightarrow$ NaN. Sau parse, bucket years_exp = 9 bất thường: chỉ 225 mẫu, median lương 8.5 triệu (thấp nhất toàn thang), và **99.6% mẫu có job_position**

= "**Chưa cập nhật**" (so với 6-10% ở bucket khác). Gần như chắc chắn là lỗi scraper (có thể nhầm cột “9 ứng viên đã ứng tuyển” thành “9 năm”).

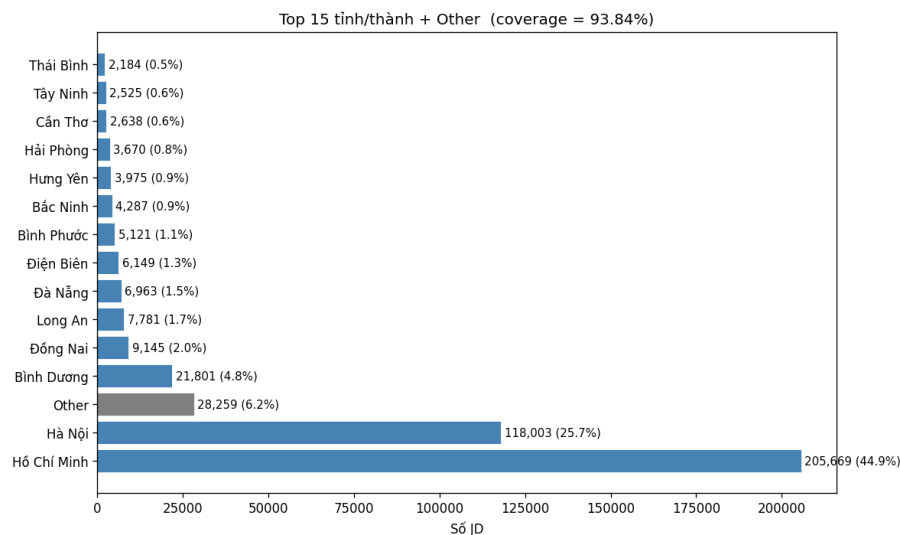
Để xử lý có nguyên tắc, nhóm dùng **rule phát hiện junk**: bucket có $n_rows < 500$ và $pos_missing_rate > 0.5$ thì set $years_exp = NaN$ cho các dòng đó. Rule fit trên train, áp nguyên cho test (loại 225 dòng train + 49 dòng test).



Bucket $years_exp$ trên train. Bucket 9.0 (đỏ) bị rule phát hiện là junk do $n=225 < 500$ và $pos_missing = 99.6\%$ (mọi bucket khác đều $< 10\%$).

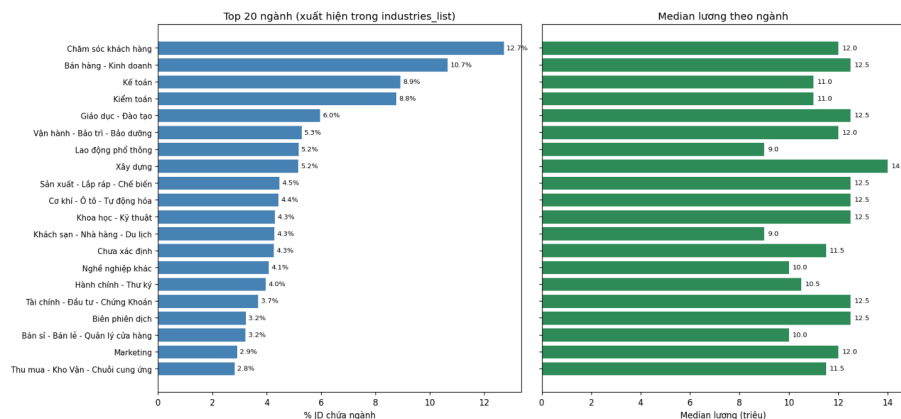
2.2.3. Tỉnh thành và ngành nghề

location thô có 215,997 giá trị unique. Nhóm dùng **regex 2 lớp**: lớp 1 match 63 tỉnh/thành chuẩn; lớp 2 (fallback) tra dictionary district → province cho HCM và HN (vì hai thành phố này chiếm 70%+ thị trường và nhiều bài chỉ ghi quận, vd. “*Quận 1*”, “*Cầu Giấy*”). Coverage đạt **93.84% train** và **93.82% test** (gần như y hệt — pattern tổng quát hoá tốt); 6.16% còn lại gom thành Other.



Top 15 tỉnh/thành theo số bản tin trên train. HCM và HN chiếm xấp xỉ 70% — phản ánh trung tâm tuyển dụng tập trung ở 2 thành phố lớn nhất.

job_industry cho phép nhiều ngành phân cách / (vd. “*Kế toán / Kiểm toán / Tài chính*”). **Không tách dấu** - vì các tên như “*Khoa học - Kỹ thuật*” là tên ngành ghép. Sau tách, có 95 ngành base; giữ **top-50** (cover 99.97%), còn lại gom Other. Khoảng 64% bản tin có 1 ngành, 35% có 2, 1% có 3 — đặc trưng multi-label thật sự.



Top 20 ngành (theo % bản tin chứa ngành) và median lương tương ứng (train). Bất động sản (20.4tr) và Ngân hàng (16.9tr) dẫn đầu; Lao động phổ thông (9.6tr) và Khách sạn-Nhà hàng (10.8tr) thấp nhất.

2.2.4. Các trường còn lại

Categorical ngắn (education_level, job_type, job_position) sau chuẩn hoá lower().strip() có cardinality 7/8/15 — phù hợp one-hot. Các cột text dài (job_title, job_description, requirements, benefits) được strip() và giữ NaN → chuỗi rỗng; không lowercase ở stage cleaning để stage 3 vectorizer xử lý. Tỷ lệ rỗng < 0.02% ở mọi cột text. Sau cleaning, tỷ lệ missing duy nhất còn lại là years_exp ≈ 2.05% — giữ NaN để stage 3 quyết định cách impute.

3. Trích xuất đặc trưng

3.1. Chiế

Stage 3 chuyển bảng dữ liệu sạch thành ma trận đặc trưng số học sparse cho sklearn/LightGBM, đạt **32,107 cột** chia làm 7 nhóm:

STT

Nhóm

Cột nguồn

Transform

Số cột

1

numeric

years_exp, year

SimpleImputer(median) + StandardScaler

2

2

one_hot

province, education_level, job_type, job_position

OneHotEncoder

94

3

industries

industries_list (split \\)

MultiLabelBinarizer

51

4

tfidf_job_title

job_title

TF-IDF (1,2)-gram, max=5k

5,000

5

tfidf_job_description

job_description

TF-IDF (1,1), max=10k

10,000

6

tfidf_requirements

requirements

TF-IDF (1,1), max=10k

9,087

7

tfidf_benefits

benefits

TF-IDF (1,1), max=10k

7,873

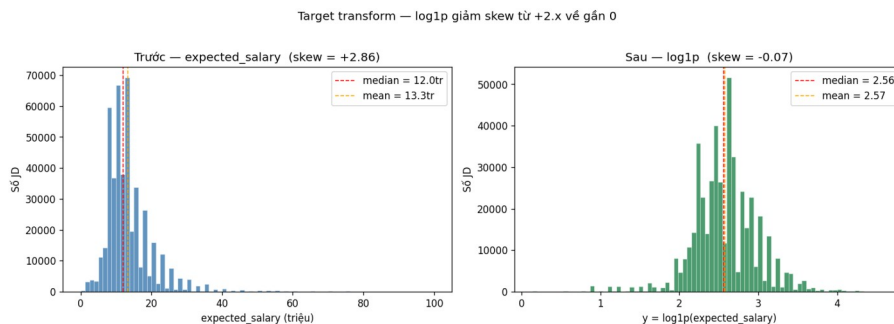
Tổng

32,107

Tất cả transformer **fit trên train**, **transform** cho test (không leak).

3.2. Biến đổi target

expected_salary lệch phải mạnh (skew $\approx +2.5$). Trong hồi quy với MSE, outlier ở đuôi xa sẽ chi phối tối ưu. Áp $y = \log_{10}(\text{expected_salary})$ kéo phân phối về gần normal (skew giảm về gần 0), giúp Ridge/Lasso/LightGBM đều ổn định hơn. Khi báo cáo metric, nhóm nghịch đảo qua expm1 về thang triệu VNĐ để dễ giải nghĩa.



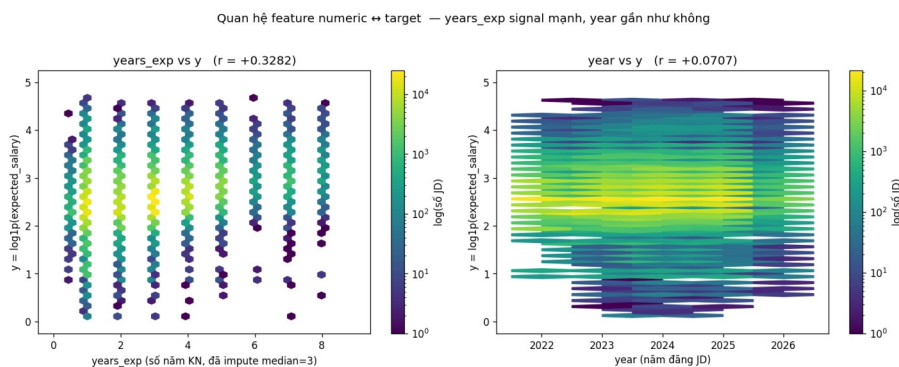
Phân phối target trước và sau log1p. Skew giảm từ +2.x về gần 0.

3.3. Đặc trưng numeric, categorical, multi-label

Numeric: years_exp impute median (=3.0); year không NaN. Cả hai chuẩn hoá StandardScaler() (center + scale). **Lưu ý quan trọng:** phiên bản đầu dùng with_mean=False giữ sparse, nhưng year sau scale ~ 1730 (vì year/std $\approx 2024/1.17$) làm Lasso-SGD diverge ngay. Center giải quyết triệt để mà chỉ thêm 0.001% nnz.

Categorical: 4 cột encode bằng OneHotEncoder(handle_unknown='ignore') $\rightarrow$ 94 cột. Tham số ignore xử lý category lạ trong test mà không lỗi.

Multi-label: industries_list split $| \rightarrow$ MultiLabelBinarizer $\rightarrow$ 51 cột (50 ngành top + Other). Mỗi mẫu có 1 ở các cột ngành mình thuộc về.



Quan hệ feature numeric với target (hexbin train). years_exp có signal rõ ($r = +0.33$); year gần như không có ($r = +0.07$) — gợi ý sớm về year ablation ở Stage 4.

3.4. Đặc trưng TF-IDF cho text

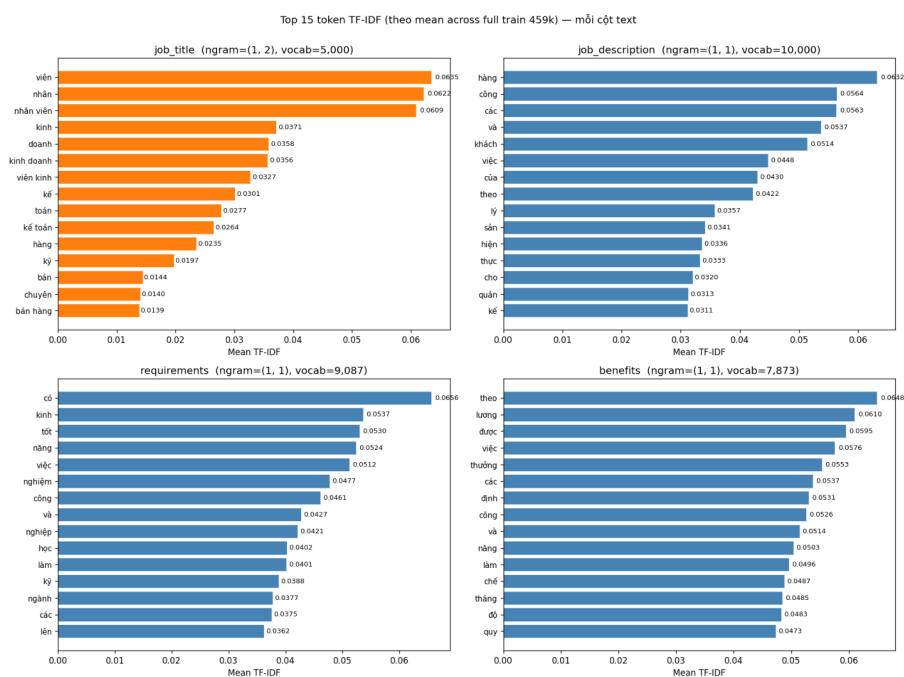
Đây là phần đông cột nhất (32k/32.1k cột). Nhóm **vectorize riêng từng cột rồi hstack** thay vì concat trước rồi vectorize 1 lần — để giữ phân biệt giữa “Python” trong requirements (yêu cầu kỹ năng) vs benefits (môi trường công nghệ).

Hai bộ tham số được dùng:

job_title (text ngắn, signal đậm đặc): max_features=5000, **ngram=(1,2)** bắt cụm chức danh (“kế toán trưởng”, “senior engineer”), min_df=5.

job_description, requirements, benefits (text dài): max_features=10000, **unigram**, min_df=10 (bigram trên text dài tồn vocab mà không thêm signal).

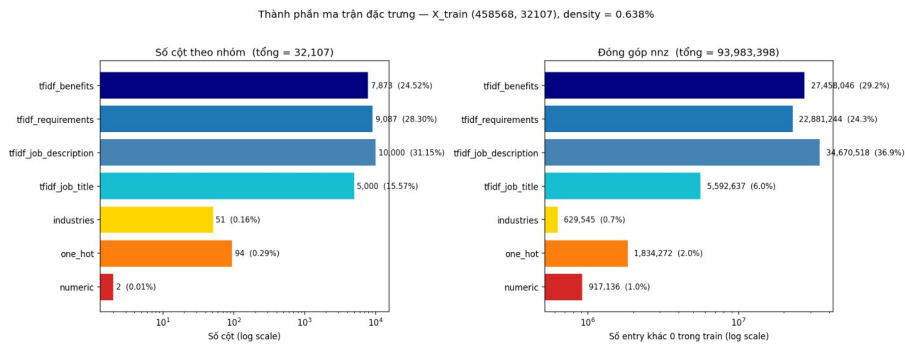
Tham số strip_accents=None **giữ dấu tiếng Việt** (dấu mang nghĩa: “thầy giáo” $\neq$ “thay giao”); sublinear_tf=True áp $\log(\text{tf})+1$ giảm tác động của từ lặp nhiều lần trong cùng văn bản.



Top 15 token TF-IDF (mean trên toàn train) cho 4 cột text. job_title (cam) dùng bigram bắt cụm chức danh; 3 cột text dài (xanh) dùng unigram.

3.5. Hợp nhất và lưu artifact

Bảy nhóm cột nối ngang qua scipy.sparse.hstack theo thứ tự cố định. Ma trận cuối: **X_train shape (458,568 × 32,107), nnz = 93.98M, density = 0.638%**; X_test (114,773 × 32,107). TF-IDF text chiếm ~99.5% cột nhưng chỉ ~80% nnz (sparse per row); numeric/OHE/multi-label dense per row nhưng tổng cộng < 0.5% cột.



Cấu phần ma trận X_{train} — bên trái: số cột; bên phải: nnz contribution.

Lưu vào features/: $X_{\{\text{train}, \text{test}\}.npz}$ (CSR sparse), $y_{\text{train}}.npy$ (log1p), transformers.joblib, meta.json (group indices + tham số), feature_names.txt (32,107 tên đặc trưng).

4. Mô hình hoá dữ liệu

4.1. Tổng quan và protocol đánh giá

Nhóm thử nghiệm **5 nhóm mô hình** với vai trò khác nhau: **Baseline group-mean** (sàn so sánh), **Ridge L2** (linear baseline cho sparse text), **Lasso-SGD L1** (feature selection), **LightGBM với Optuna** (tree boosting + Bayesian tuning), và **Ensemble weighted** (kết hợp 3 model trên).

Protocol 3 bước chống overfit:

Stratified subsample 150,000 mẫu từ train, phân tầng theo ngành đầu trong industries_list.

CV chọn hyperparam trên sample: 5-fold cho Ridge/Lasso, 3-fold cho LightGBM bên trong Optuna (cân bằng số trial $\times$ thời gian).

Refit best hyperparam trên FULL train (~459k) $\rightarrow$ evaluate test (~114k) **đúng một lần duy nhất**.

Mọi metric (MAE, RMSE, R^2) tính ở thang **triệu VNĐ** (sau expm1). R^2 log space báo cáo bổ sung.

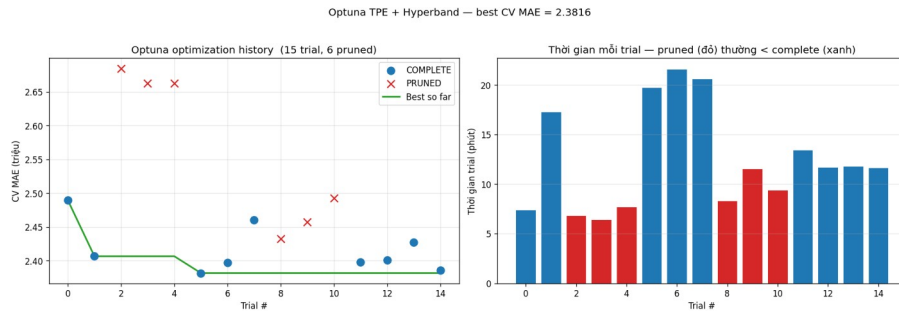
4.2. Cơ sở lý thuyết các mô hình

Baseline group-mean tra cứu median lương theo cặp (ngành đầu, năm KN) fit trên train với fallback hierarchy — non-parametric, capture được signal chính. **Ridge** [9] (L2 regularization) tối ưu closed-form qua solver sparse_cg, grid $\alpha \in \{0.1, 1, 10, 100\}$. **Lasso** [1] (L1) cho feature selection tự động; nhóm dùng SGDRegressor(penalty='l1') thay sklearn.Lasso vì scale tốt hơn trên sparse 459k $\times$ 32k, grid $\alpha \in \{1e-5, 1e-4, 1e-3, 1e-2\}$. **LightGBM** [2] là gradient boosting decision trees với GOSS + EFB, hỗ trợ sparse trực tiếp; thay grid search, nhóm dùng **Optuna** [3] với **TPE sampler** [5] và **Hyperband pruner** [10], tinh chỉnh đồng thời **7 hyperparam**: num_leaves (15-127), learning_rate

(0.01-0.15, log), min_child_samples (5-100), feature_fraction (0.6-1.0), bagging_fraction (0.6-1.0), reg_alpha/reg_lambda (1e-8 - 10, log). Budget 15 trial $\times$ 3-fold CV, early_stopping(30). **Ensemble weighted** dùng trọng số $w_i = (1 / MAE_i) / \sum_j (1 / MAE_j)$ — model tốt hơn weight cao hơn.

4.3. Tinh chỉnh LightGBM với Optuna

Optuna chạy **15 trial $\times$ 3-fold CV** trên 150k sample, mất **185.7 phút** (~3.1h) trên 16 core; **6 trial bị pruned** (40%) — tiết kiệm 30-40% compute so với full search.



Optuna optimization history. Bên trái: MAE từng trial (xanh = COMPLETE, đỏ = PRUNED), đường xanh lá = best so far; bên phải: thời gian trial.

Best params: num_leaves=125 (chạm trần range, có thể nói rộng), learning_rate=0.068, min_child_samples=98 (tránh overfit bucket nhỏ), feature_fraction=0.607, bagging_fraction=0.994, reg_alpha=0.253, reg_lambda=2e-8, n_estimators=550 (median best_iter $\times$ 1.1). Best CV MAE = **2.382**.

4.4. Kết qu

Mỗi model refit FULL train (459k) → predict test (114,773) **đúng một lần**.

Mô hình

MAE (triệu)

RMSE (triệu)

R² (M)

R² (log)

Baseline (group-mean)

3.926

6.095

0.187

0.198

Ridge ($\alpha=10$)

2.573

4.568

0.543

0.650

Lasso-SGD ($\alpha=1e-5$, nnz=3,614)

2.668

4.720

0.512

0.627

LightGBM (Optuna, leaves=125)

2.199

4.097

0.632

0.718

LightGBM (no year)

2.203

4.111

0.630

0.717

Ensemble (weighted)

2.400

4.362

0.583

0.687

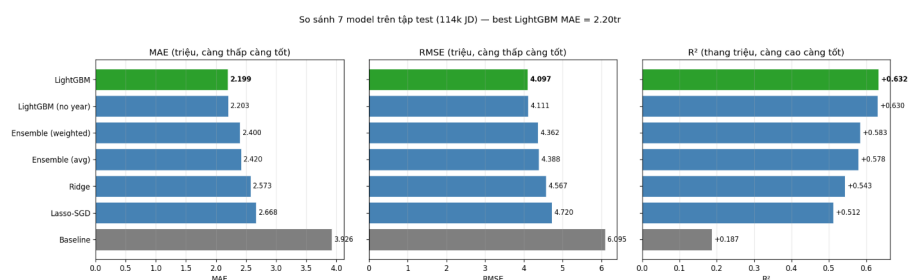
Ensemble (simple avg)

2.420

4.388

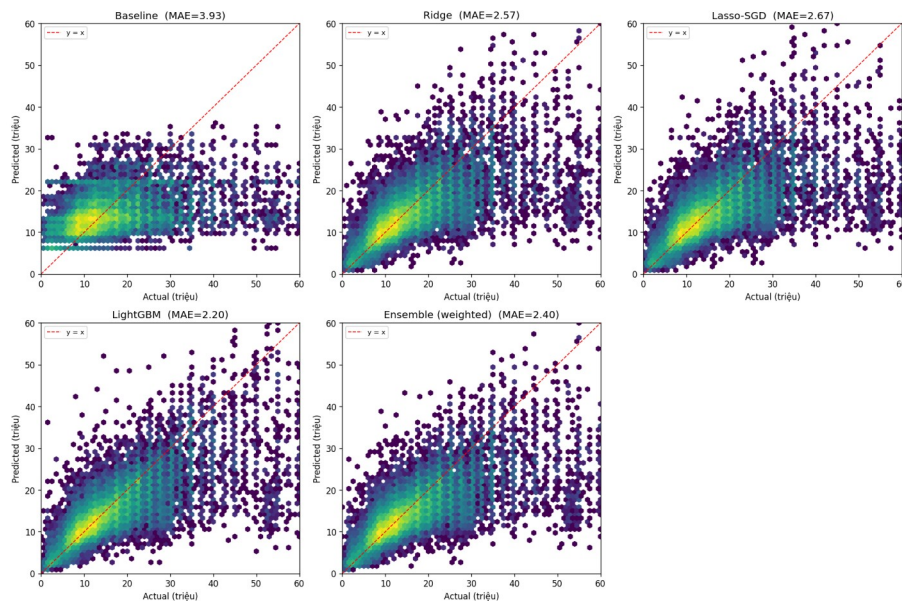
0.578

0.683

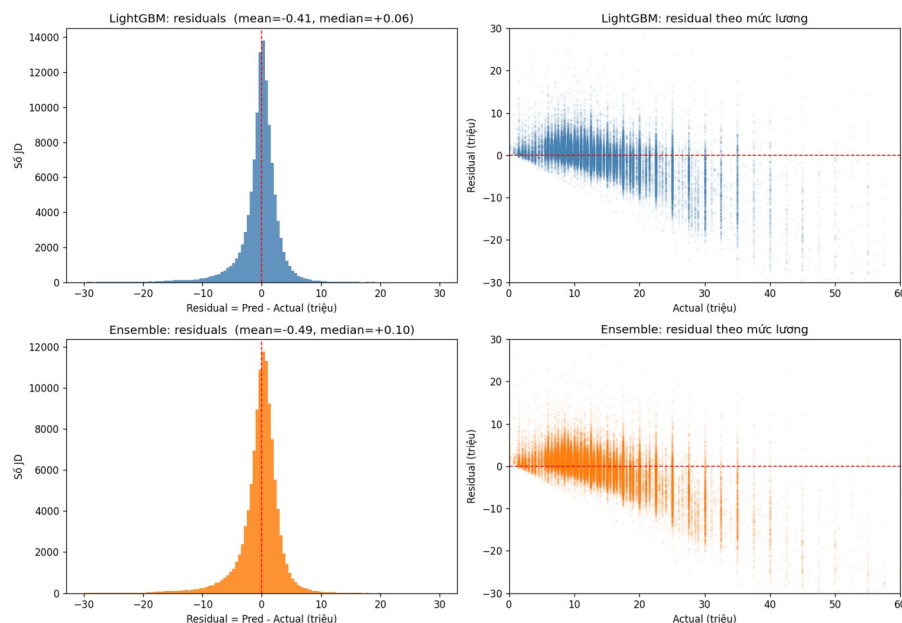


So sánh 7 mô hình trên test (114k JD). LightGBM (xanh lá) MAE thấp nhất 2.20 — cải thiện 44% so với baseline. Ensemble không vượt được LightGBM đơn lẻ.

Quan sát chính: (i) **LightGBM thắng tuyệt đối** — MAE 2.20, giảm 44% so với baseline, 15% so với Ridge, 18% so với Lasso; (ii) **Ridge $\approx$ Lasso-SGD** với MAE cách nhau 0.1 triệu, Lasso giữ chỉ $3,614 / 32,107 = 11.3\%$ hệ số khác 0 (xác nhận phần lớn TF-IDF là noise); (iii) **Ensemble TỆ hơn LightGBM $\sim 9\%$** — sẽ phân tích ở Mục 4.7.



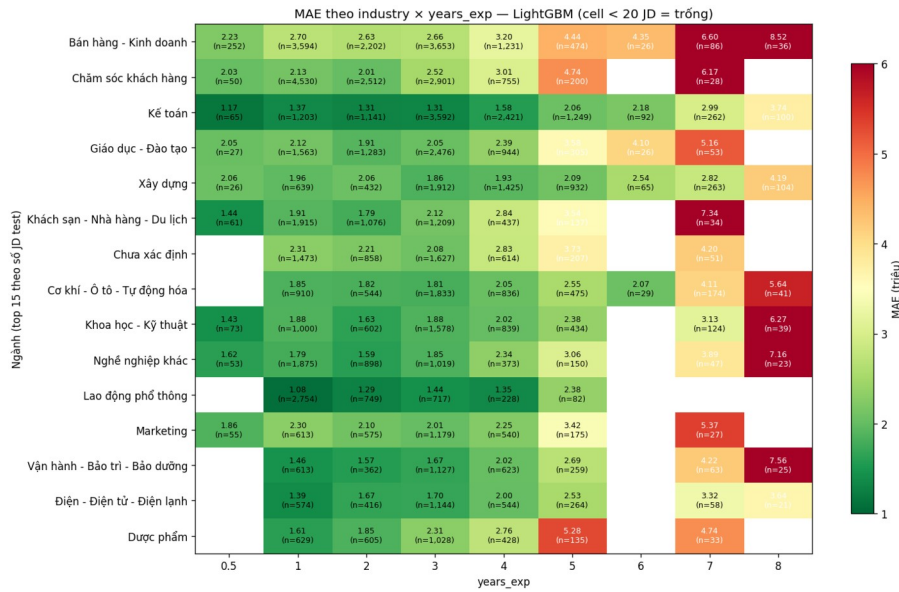
Dự đoán vs thực tế cho 5 mô hình (hexbin density). Baseline rời rạc theo các cặp (ngành, KN); các model còn lại dày đặc dọc theo $y=x$, LightGBM dày nhất quanh vùng 8-25 triệu.



Residual của LightGBM và Ensemble. Histogram (trái) cho thấy phân phối lệch nhẹ về phía dưới; scatter (phải) cho thấy heteroscedasticity — sai số tăng theo mức lương, đặc biệt > 40 triệu.

4.5. Phân tích lỗi và year ablation

Sai số phân tầng theo ngành × kinh nghiệm

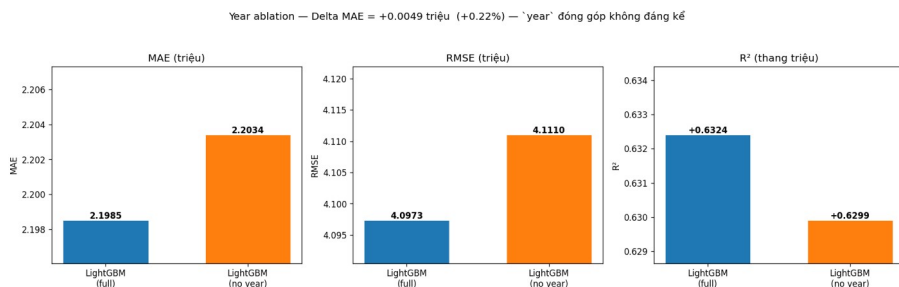


MAE theo (ngành × years_exp) cho LightGBM trên top 15 ngành test. Đỏ = MAE cao, xanh = MAE thấp.

Phát hiện chính: **MAE tăng theo năm KN** ở hầu hết ngành (0.5-3 năm: 1.5-2.5tr; 7-8 năm: 4-7tr) — phù hợp heteroscedasticity ở residual scatter. **Bất động sản** MAE cao nhất (~6-7tr) do lương commission-based phân tán mạnh; **Lao động phổ thông** và **Kế toán** thấp nhất (~1.2-1.7tr) do lương cố định.

Year ablation — year có ý nghĩa không?

Để kiểm chứng đóng góp của year (gợi ý yếu từ $r = +0.07$ ở Stage 3), nhóm zero hết cột year trong X (qua `scipy.sparse CSC zero`), refit LightGBM với chính best params từ Optuna.



Year ablation — LightGBM full vs no year. Delta MAE = +0.005 triệu (+0.22%) — year đóng góp không đáng kể.

Delta MAE chỉ +0.005 triệu (+0.22%). Giải thích: dữ liệu chỉ trải 5 năm (2022-2026), lạm phát giai đoạn này không lớn, và ngành/kinh nghiệm/chức danh đã capture phần lớn signal.

Feature importance

Phân tích top 30 feature theo Ridge coefficient và LightGBM gain (xem chi tiết trong figures/feature_importance.png): cả hai mô hình đều thiên về **TF-IDF text** và **industries** ở top — text là nguồn signal chính, vượt trội so với categorical đơn lẻ. Ridge bắt được cả tín hiệu dương (từ khoá lương cao) và âm (từ khoá lương thấp); LightGBM gain luôn dương do bản chất tree splitting. years_exp có gain đáng kể ở LightGBM — phù hợp $r = +0.33$.

4.6. Vì sao Ensemble không cải thiện — phân tích sâu

Đây là **phát hiện đáng chú ý nhất** ở Stage 4. Theo lý thuyết, ensemble các model lỗi không tương quan hoàn hảo sẽ giảm noise và cho MAE thấp hơn. Thực nghiệm cho kết quả **ngược lại**: ensemble MAE = 2.40, cao hơn LightGBM 0.20 triệu (+9%).

Nguyên nhân 1
— Trọng số
1/MAE quá nhẹ
tay:

Model

MAE

1/MAE

Weight

Ridge

2.573

0.389

31.9%

Lasso-SGD

2.668

0.375

30.8%

LightGBM

2.199

0.455

37.3%

LightGBM tốt hơn 17% nhưng chỉ thêm ~5 điểm phần trăm weight. Hai model linear cộng lại chiếm **62.7%** — kéo dự đoán về phía linear vốn yếu hơn.

Nguyên nhân 2 — Hai model linear có lỗi gần như y hệt:

Correlation error
trên test:

Ridge

Lasso-SGD

LightGBM

Ridge

1.000

0.985

0.913

Lasso-SGD

0.985

1.000

0.906

LightGBM

0.913

0.906

1.000

Ridge và Lasso-SGD sai gần y hệt ($r = 0.985$) — gộp 2 model nhìn giống nhau $\approx$ **double-counting góc nhìn linear**. Ensemble thực chất chỉ có 2 “ý kiến” độc lập (linear vs tree), linear chiếm tỉ trọng lớn $\rightarrow$ kéo chất lượng xuống.

Quyết định cuối: nhóm **không sử dụng ensemble** trong pipeline triển khai. Model production = **LightGBM đơn lẻ** với MAE 2.20 triệu. Ensemble được giữ trong báo cáo như một **negative result có giá trị** — minh chứng rằng kết hợp model không luôn cải

thiện, và việc chọn weighting scheme đơn giản có thể làm tệ đi nếu model thành phần không đủ đa dạng.

5. Kết luận

5.1. Tổng kết kết quả

Nhóm đã xây dựng thành công pipeline 4 giai đoạn dự đoán lương kỳ vọng từ bản mô tả công việc tiếng Việt. Trên tập test độc lập 114,773 mẫu, mô hình tốt nhất (LightGBM với Optuna) đạt: **MAE = 2.199 triệu**, **RMSE = 4.097 triệu**, **$R^2 = 0.632$** . So với baseline group-mean (MAE 3.93), pipeline cải thiện **44%** sai số. So với lương trung bình toàn tập (13.32 triệu), MAE 2.20 tương đương khoảng **16.5% sai số tương đối** — kết quả tốt cho một bài toán hồi quy trên dữ liệu văn bản không cấu trúc.

5.2. Những phát hiện đáng chú ý

Năm đăng không đóng góp: ablation cho thấy loại bỏ year chỉ tăng MAE 0.005 triệu (+0.22%). Ý nghĩa thực tiễn: model có thể triển khai cho năm mới (2027+) mà không lo drift do năm, miễn là phân phối feature khác không đổi đáng kể.

Ensemble không cải thiện: dù lý thuyết ensemble giảm MAE, thực nghiệm cho MAE cao hơn 9%. Hai nguyên nhân: trọng số 1/MAE quá đồng đều (~31/31/37%) khiến LightGBM không thống trị; lỗi 2 model linear tương quan rất cao ($r = 0.985$) — gộp lại không thêm thông tin. Minh chứng rằng ensemble cần xét **độ đa dạng** của model thành phần, không chỉ chất lượng từng cái.

5.3. Hạn chế

Search space LightGBM chạm trần: num_leaves best = 125, gần biên trên 127 — mở rộng (vd. 15-255) có thể tìm model tốt hơn nhưng tốn vài giờ. (2) **Chưa thử neural model**: PhoBERT [7] fine-tune có thể vượt LightGBM nhờ ngữ nghĩa contextual nhưng chi phí cao hơn nhiều. (3) **TF-IDF là bag-of-words**: không phân biệt “3 năm kinh nghiệm Python” với “không cần kinh nghiệm Python”. (4) **Heteroscedasticity ở vùng lương cao**: MAE tăng theo mức lương — có thể cải thiện bằng quantile regression.

5.4. Hướng phát triển

Ba hướng khả thi: (1) **Mở rộng search space Optuna** (50-100 trial, num_leaves $\in [15, 255]$) — kỳ vọng giảm thêm MAE 0.05-0.10 triệu. (2) **Fine-tune PhoBERT** cho job_description và requirements thay TF-IDF. (3) **Quantile regression** dự đoán P10/P50/P90 thay vì point estimate — cung cấp confidence interval cho mỗi prediction, hữu ích cho ứng dụng phát hiện outlier lương.

Tổng kết, dự án đã đạt mục tiêu: xây dựng được mô hình dự đoán lương có độ chính xác tốt (MAE 2.2 triệu trên 114k mẫu test), đồng thời ghi nhận các phát hiện về phương pháp luận (year-effect và ensemble) có giá trị tham khảo cho nghiên cứu sau.

6. Tài liệu tham khảo

- [1] **Tibshirani, R.** (1996). *Regression shrinkage and selection via the Lasso*. Journal of the Royal Statistical Society: Series B (Methodological), 58(1), 267-288.
- [2] **Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y.** (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Advances in Neural Information Processing Systems (NeurIPS) 30, 3146-3154. URL: <https://github.com/microsoft/LightGBM>
- [3] **Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M.** (2019). *Optuna: A Next-generation Hyperparameter Optimization Framework*. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2623-2631. URL: <https://optuna.org>
- [4] **tinixai** (2024). *Vietnamese Job Descriptions Dataset* (606,878 bản tin tuyển dụng tiếng Việt, 2022-2026). Hugging Face Hub. URL: <https://huggingface.co/datasets/tinixai/vietnamese-job-descriptions>
- [5] **Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B.** (2011). *Algorithms for Hyperparameter Optimization*. Advances in Neural Information Processing Systems (NeurIPS) 24, 2546-2554.
- [6] **Pedregosa, F., Varoquaux, G., Gramfort, A., et al.** (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. URL: <https://scikit-learn.org>
- [7] **Nguyen, D. Q., & Nguyen, A. T.** (2020). *PhoBERT: Pre-trained language models for Vietnamese*. Findings of the Association for Computational Linguistics: EMNLP 2020, 1037-1042. URL: <https://github.com/VinAIRResearch/PhoBERT>
- [8] **Spärck Jones, K.** (1972). *A Statistical Interpretation of Term Specificity and Its Application in Retrieval*. Journal of Documentation, 28(1), 11-21.
- [9] **Hoerl, A. E., & Kennard, R. W.** (1970). *Ridge Regression: Biased Estimation for Nonorthogonal Problems*. Technometrics, 12(1), 55-67.
- [10] **Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A.** (2017). *Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization*. Journal of Machine Learning Research, 18(185), 1-52.